

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
print("Libraries imported successfully")
```

Libraries imported successfully

```
In [3]: # Load the dataset
df = pd.read_csv('customer_segmentation_data.csv')

# display the first 5 rows
df.head()
```

Out[3]:

	Customer ID	Age	Gender	Marital Status	Education Level	Geographic Information	Occupation
0	84966	23	Female	Married	Associate Degree	Mizoram	Entrepreneur
1	95568	26	Male	Widowed	Doctorate	Goa	Manager
2	10544	29	Female	Single	Associate Degree	Rajasthan	Entrepreneur
3	77033	20	Male	Divorced	Bachelor's Degree	Sikkim	Entrepreneur
4	88160	25	Female	Separated	Bachelor's Degree	West Bengal	Manager

```
In [4]: # Check for Missing values and data types
print("\n--- Data Information ---")
df.info()
```

```
--- Data Information ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 53503 entries, 0 to 53502
Data columns (total 20 columns):
#   Column                                                                 Non-Null Count  Dtype
---  -
0   Customer ID                                                            53503 non-null  int64
1   Age                                                                    53503 non-null  int64
2   Gender                                                                53503 non-null  object
3   Marital Status                                                         53503 non-null  object
4   Education Level                                                        53503 non-null  object
5   Geographic Information                                                 53503 non-null  object
6   Occupation                                                            53503 non-null  object
7   Income Level                                                          53503 non-null  int64
8   Behavioral Data                                                       53503 non-null  object
9   Purchase History                                                      53503 non-null  object
10  Interactions with Customer Service 53503 non-null  object
11  Insurance Products Owned                                              53503 non-null  object
12  Coverage Amount                                                       53503 non-null  int64
13  Premium Amount                                                        53503 non-null  int64
14  Policy Type                                                           53503 non-null  object
15  Customer Preferences                                                  53503 non-null  object
16  Preferred Communication Channel 53503 non-null  object
17  Preferred Contact Time                                                53503 non-null  object
18  Preferred Language                                                    53503 non-null  object
19  Segmentation Group                                                    53503 non-null  object
dtypes: int64(5), object(15)
memory usage: 8.2+ MB
```

```
In [5]: print("Check for missing values")
missing = df.isnull().sum()
print(missing)
```

```
Check for missing values
Customer ID          0
Age                  0
Gender                0
Marital Status       0
Education Level      0
Geographic Information 0
Occupation           0
Income Level         0
Behavioral Data      0
Purchase History     0
Interactions with Customer Service 0
Insurance Products Owned 0
Coverage Amount      0
Premium Amount       0
Policy Type          0
Customer Preferences 0
Preferred Communication Channel 0
Preferred Contact Time 0
Preferred Language    0
Segmentation Group   0
dtype: int64
```

```
In [18]: print("\n" + "="*25)
print("Check Duplicates Rows")
print("="*25)
duplicates = df.duplicated().sum()
print("Number of Duplicates rows:", duplicates)
```

```
=====
Check Duplicates Rows
=====
Number of Duplicates rows: 0
```

```
In [17]: print("\n" + "="*25)
print("Gender vs Marital Status")
print("="*25)
```

```
=====
Gender vs Marital Status
=====
```

```
In [12]: # Gender vs Marital Status
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Gender Distribution
ax = sns.countplot(data=df, x='Gender', palette='Set2', ax=axes[0])
axes[0].set_title('Distribution by Gender', fontweight='bold')
for container in ax.containers: ax.bar_label(container)

# Marital Status Distribution
ax = sns.countplot(data=df, x='Marital Status', palette='Set3', ax=axes[1])
axes[1].set_title('Distribution by Marital Status', fontweight='bold')

for container in ax.containers: ax.bar_label(container)

plt.tight_layout() # Clean the margins
plt.show()
```

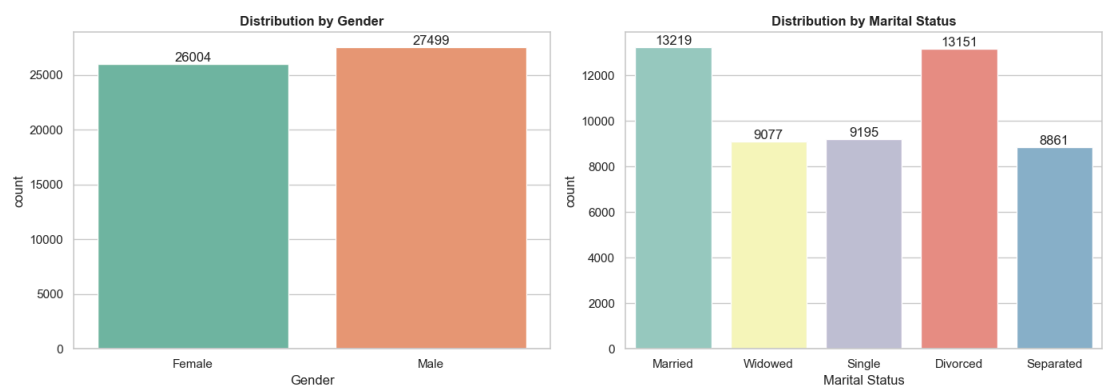
C:\Users\vivek\AppData\Local\Temp\ipykernel_13744\3176868789.py:5:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.countplot(data=df, x='Gender', palette='Set2', ax=axes[0])
C:\Users\vivek\AppData\Local\Temp\ipykernel_13744\3176868789.py:10:  
FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.countplot(data=df, x='Marital Status', palette='Set3',
ax=axes[1])
```



```
In [19]: print("\n" + "="*25)
print("Age and Income")
print("="*25)
```

```
=====
Age and Income
=====
```

```

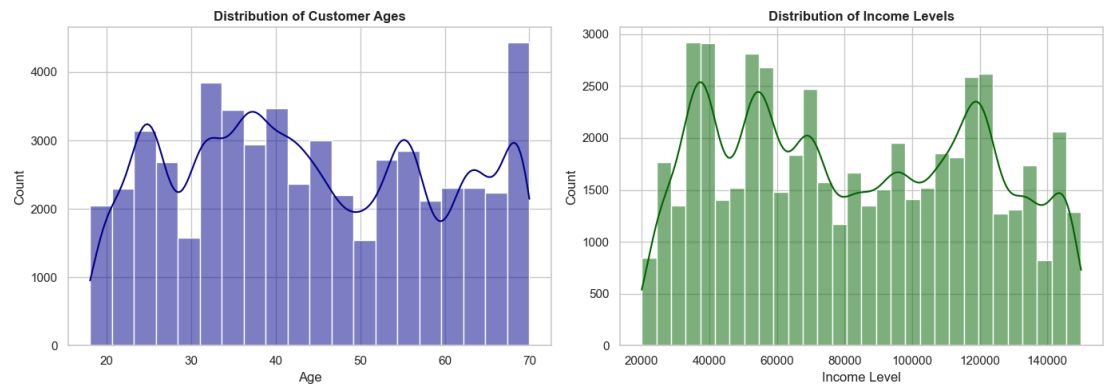
In [26]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Age Distribution
sns.histplot(data=df, x='Age', bins=20, kde=True, color='darkblue', ax=axes[0])
axes[0].set_title('Distribution of Customer Ages', fontweight='bold')

# Income Distribution
sns.histplot(data=df, x='Income Level', bins=30, kde=True, color='darkgreen',
ax=axes[1])
axes[1].set_title('Distribution of Income Levels', fontweight='bold')

plt.tight_layout()
plt.show()

```



```

In [31]: from sklearn.cluster import KMeans
          from sklearn.preprocessing import StandardScaler

          # Choose the features we want to cluster by
          X = df[['Income Level', 'Premium Amount']].values

          # Scale the data
          scaler = StandardScaler()
          X_scaled = scaler.fit_transform(X)

          print("Data prepared and scaled successfully. Calculating the optimal
          clusters....\n")

          # Calculate Within-Cluster Sum of Square for K values from 1 to 10
          wcss = []

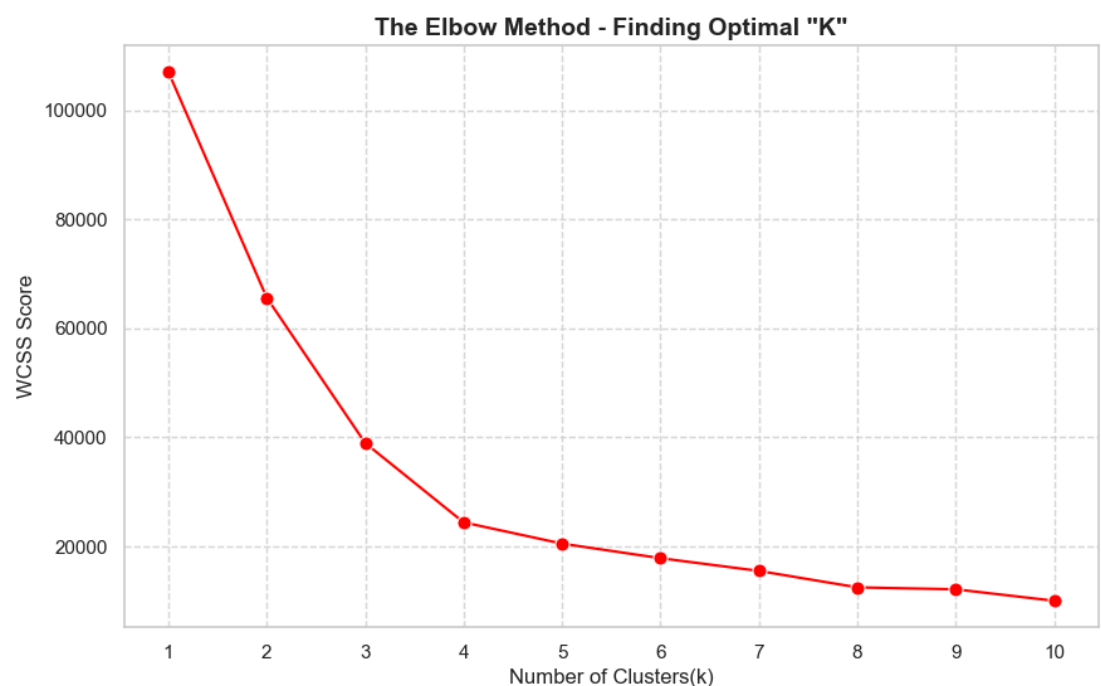
          for i in range(1, 11):
              # k-means++ ensures the algorithm starts efficiently
              kmeans = KMeans(n_clusters = i, init='k-means++', random_state=42)
              kmeans.fit(X_scaled)
              wcss.append(kmeans.inertia_)    # inertia_ is the WCSS score

          # plot Elbow Graph
          plt.figure(figsize=(10, 6))
          sns.lineplot(x=range(1, 11), y=wcss, marker='o', color='red', markersize=8)

          plt.title('The Elbow Method - Finding Optimal "K"', fontweight='bold',
          fontsize=14)
          plt.xlabel('Number of Clusters(k)')
          plt.ylabel('WCSS Score')
          plt.xticks(range(1, 11))
          plt.grid(True, linestyle='--', alpha=0.7)
          plt.show()

```

Data prepared and scaled successfully. Calculating the optimal clusters....



In [35]:

```
# Initialize and build the final K-Means model with K=4
kmeans = KMeans(n_clusters=4, init='k-means++', random_state=42)

# Predict which cluster each customer belongs to
y_kmeans = kmeans.fit_predict(X_scaled)

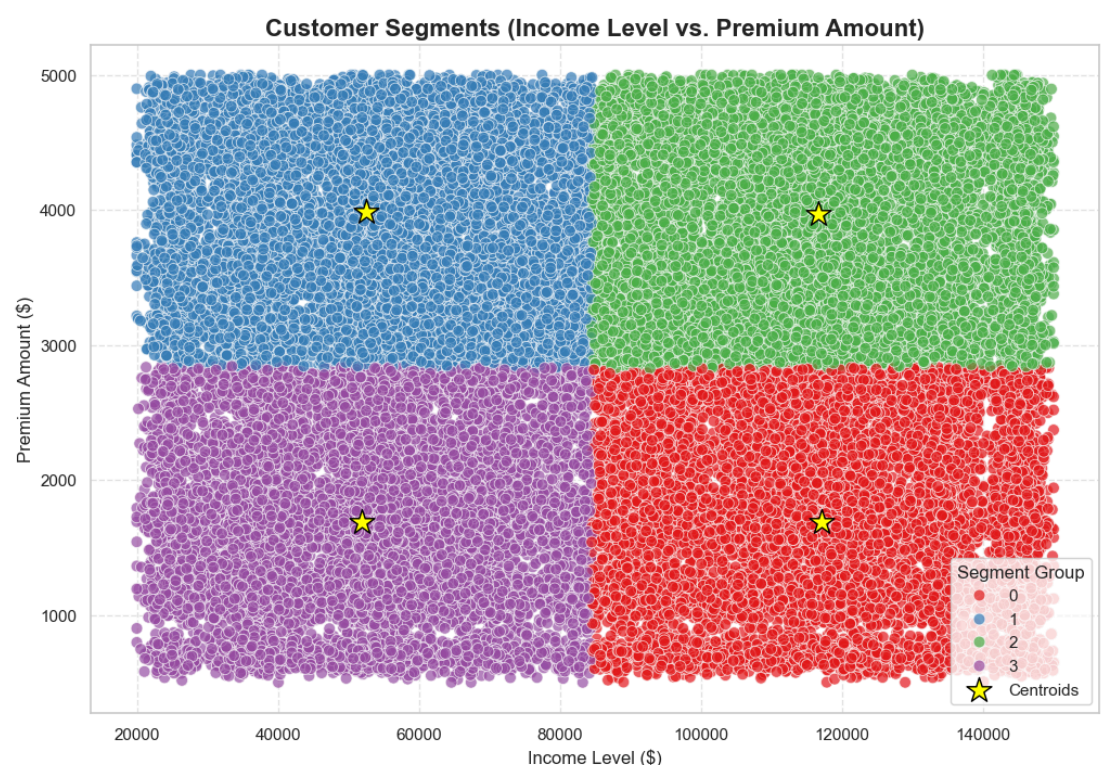
# Add new cluster labels back to our original dataframe
df['Customer Segment'] = y_kmeans

# Visualize the Clusters
plt.figure(figsize=(12, 8))

# Create a Scatter plot coloring points by their assigned cluster
sns.scatterplot(
    x = df['Income Level'],
    y = df['Premium Amount'],
    hue = df['Customer Segment'],
    palette = 'Set1',
    s = 60,
    alpha = 0.7
)

# Plot the Centroids (The mathematical center of each cluster)
# We have to reverse the scaling we did earlier so the centroids plot correctly on
the original axes
centroids = scaler.inverse_transform(kmeans.cluster_centers_)
plt.scatter(centroids[:, 0], centroids[:, 1], s=300, c='yellow',
            edgecolor='black', marker='*', label='Centroids')

plt.title('Customer Segments (Income Level vs. Premium Amount)',
          fontweight='bold', fontsize=16)
plt.xlabel('Income Level ($)')
plt.ylabel('Premium Amount ($)')
plt.legend(title='Segment Group')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```



Executive Summary & Business Recommendations

Objective: Segment the 53,000+ customer base using K-Means clustering to identify distinct purchasing behaviors and optimize targeting strategies.

Methodology: Extracted 'Income Level' and 'Premium Amount' as key financial indicators. The data was standardized using `StandardScaler` to ensure accurate distance calculations. The Elbow Method was utilized to determine the optimal number of clusters, which was identified as $K=4$.

Cluster Insights:

Based on the K-Means algorithm, our customer base naturally divides into 4 distinct segments:

- #### 1. **Low Income, Low Premium (Budget Conscious):** Customers who likely purchase minimum coverage policies. **Strategy:** Market basic, high-value, and family-bundle policies.
- #### 2. **Low Income, High Premium (Over-extended/High Risk):** Customers spending a large portion of their income on insurance. **Strategy:** Investigate for potential churn risk or offer flexible payment plans.
- #### 3. **High Income, Low Premium (Untapped Potential):** Customers with high spending power but low current investment. **Strategy:** Prime targets for cross-selling, premium upgrades, and comprehensive coverage plans.
- #### 4. **High Income, High Premium (VIPs):** The most profitable segment. **Strategy:** Focus on retention, elite customer service, and loyalty rewards to maintain this critical revenue stream.

In []: